

Secure Coding Review Report Project

Simple Login & Notes App (Flask)

Reviewer: Jannatul Ferdaus Oishi

Task: CodeAlpha Cyber Security Internship — Task 3: Secure Coding Review

1. Project Overview

The reviewed application is a small Flask web app (app.py) that provides a login form backed by a SQLite users table, a dashboard page shown after login, and a /run endpoint intended for admin/debug use.

The app was written deliberately for this review exercise, containing six common real-world coding mistakes seen in early-stage or rushed web applications.

2. Methodology

Three review methods were combined:

1. Automated static analysis using Bandit, a Python security linter, run against both the vulnerable and fixed versions (bandit -r app.py and bandit -r app_fixed.py).

2. Manual code review — reading every line of app.py to look for logic-level issues automated tools can miss (e.g. plaintext password storage, verbose error messages, missing session checks).

3. Live exploitation testing — running the app locally and reproducing each vulnerability's real-world effect to confirm actual impact, not just theoretical risk.

3. Findings Summary

#	Vulnerability	Severity	Location	Found By	Bandit ID
1	Hardcoded secret key in source code	Low	app.py:7	Bandit + Manual	B105
2	Passwords stored in plaintext in the database	High	app.py (init_db)	Manual only	—
3	SQL Injection via string-concatenated query	Medium/Critical	app.py:61	Bandit + Manual	B608

4	Verbose error message leaks raw SQL query to the client	Medium	<code>app.py</code> (login)	Manual only	—
5	Command injection via unsanitized shell execution (<code>os.popen</code>)	High/Critical	<code>app.py:105</code>	Bandit + Manual	B605
6	Flask debug mode enabled + bound to all interfaces (<code>0.0.0.0</code>)	High/Medium	<code>app.py:114</code>	Bandit + Manual	B201, B104

4. Detailed Findings

Before / After — SQL Injection Fix

Before:

Python

```
query = "SELECT * FROM users WHERE username = " + username + " AND
password = " + password + ""
cursor = db.execute(query)
```

After:

Python

```
cursor = db.execute("SELECT password FROM users WHERE username = ?",
(username,))
row = cursor.fetchone()
if row and bcrypt.checkpw(password.encode(), row[0]):
    session["user"] = username
```

Before / After — Password Storage Fix

Before:

Python

```
db.execute("INSERT OR IGNORE INTO users (id, username, password) VALUES (1, 'admin', 'admin123')")
```

After:

Python

```
hashed = bcrypt.hashpw("admin123".encode(), bcrypt.gensalt())
db.execute("INSERT OR IGNORE INTO users (id, username, password) VALUES (1, 'admin', ?)", (hashed,))
```

Before / After — Command Injection Fix

Before:

Python

```
@app.route("/run", methods=["POST"])
def run_command():
    cmd = request.form.get("cmd")
    result = os.popen(cmd).read()
    return f"<pre>{result}</pre>"
```

After:

Python

```
@app.route("/run", methods=["POST"])
def run_command():
    if "user" not in session:
        return redirect("/login")
    allowed_commands = {"date": ["date"], "whoami": ["whoami"]}
    choice = request.form.get("cmd")
    if choice not in allowed_commands:
        return "Command not allowed.", 400
```

```
result = subprocess.run(allowed_commands[choice], capture_output=True,
text=True)
return f"<pre>{result.stdout}</pre>"
```

Before / After — Hardcoded Secret Key Fix

Before:

Python

```
app.secret_key = "supersecret123"
```

After:

Python

```
app.secret_key = os.getenv("SECRET_KEY", os.urandom(24).hex())
```

(Falls back to a random key if the env var isn't set, so the app still runs securely in dev without crashing.)

Before / After — Information Leakage Fix

Before:

Python

```
if user:
    return redirect(url_for("dashboard", user=username))
else:
    return f"Login failed. Query used: {query}", 401
```

After:

Python

```
if row and bcrypt.checkpw(password.encode(), row[0]):  
    session["user"] = username  
    return redirect(url_for("dashboard"))  
else:  
    return "Invalid username or password.", 401
```

Before / After — Debug Mode Fix

Before:

python

```
app.run(debug=True, host="0.0.0.0")
```

After:

Python

```
app.run(host="127.0.0.1", debug=False)
```

6. Bandit Scan Results

6.1 Vulnerable version (app.py)

```
(venv) E:\CodeAlpha Projects\CodeAlpha_SecureCodingReview>bandit -r app.py
[main] INFO profile include tests: None
[main] INFO profile exclude tests: None
[main] INFO cli include tests: None
[main] INFO cli exclude tests: None
[main] INFO running on Python 3.11.9
Run started:2026-07-11 10:56:16.350687+00:00

Test results:
>> Issue: [B105:hardcoded_password_string] Possible hardcoded password: 'supersecret123'
Severity: Low Confidence: Medium
CWE: CWE-259 (https://cwe.mitre.org/data/definitions/259.html)
More Info: https://bandit.readthedocs.io/en/1.9.4/plugins/b105\_hardcoded\_password\_string.html
Location: .\app.py:7:17
6 # --- VULNERABILITY 1: Hardcoded secret key ---
7 app.secret_key = "supersecret123"
8 # The secret key is stored directly in the source code. If someone gets access to our GitHub repository, anyone can steal it and potentially forge sessions.

>> Issue: [B608:hardcoded_sql_expressions] Possible SQL injection vector through string-based query construction.
Severity: Medium Confidence: Low
CWE: CWE-89 (https://cwe.mitre.org/data/definitions/89.html)
More Info: https://bandit.readthedocs.io/en/1.9.4/plugins/b608\_hardcoded\_sql\_expressions.html
Location: .\app.py:61:16
60 # --- VULNERABILITY 3: SQL Injection (string concatenation, not parameterized) ---
61 query = "SELECT * FROM users WHERE username = '' + username + '' AND password = '' + password + ''"
62 # An attacker can bypass the login.Because '1'='1' is always true, the attacker may log in without the real password.

>> Issue: [B605:start_process_with_a_shell] Starting a process with a shell, possible injection detected, security issue.
Severity: High Confidence: High
CWE: CWE-78 (https://cwe.mitre.org/data/definitions/78.html)
More Info: https://bandit.readthedocs.io/en/1.9.4/plugins/b605\_start\_process\_with\_a\_shell.html
Location: .\app.py:105:13
104 import os
105 result = os.popen(cmd).read()
106 return f"<pre>{result}</pre>"

>> Issue: [B201:flask_debug_true] A Flask app appears to be run with debug=True, which exposes the Werkzeug debugger and allows the execution of arbitrary code.
Severity: High Confidence: Medium
CWE: CWE-94 (https://cwe.mitre.org/data/definitions/94.html)
More Info: https://bandit.readthedocs.io/en/1.9.4/plugins/b201\_flask\_debug\_true.html
Location: .\app.py:114:4
113 # --- VULNERABILITY 6: Debug mode enabled (exposes interactive debugger) ---
```

Code scanned:

```
Total lines of code: 60
Total lines skipped (#nosec): 0
```

Run metrics:

```
Total issues (by severity):
  Undefined: 0
  Low: 1
  Medium: 2
  High: 2
Total issues (by confidence):
  Undefined: 0
  Low: 1
  Medium: 3
  High: 1
```

```
Files skipped (0):
```

Bandit automatically caught **5 of the 6** known issues.

6.2 Vulnerable version (app_fixed.py)

```
Files skipped (0):

E:\CodeAlpha_Projects\CodeAlpha_SecureCodingReview\bandit -r app_fixed.py
[main] INFO profile include tests: None
[main] INFO profile exclude tests: None
[main] INFO cli include tests: None
[main] INFO cli exclude tests: None
[main] INFO running on Python 3.11.9
Run started:2026-07-11 11:41:47.552892+00:00

Test results:
>> Issue: [B404:blacklist] Consider possible security implications associated with the subprocess module.
Severity: Low Confidence: High
CWE: CWE-78 (https://cwe.mitre.org/data/definitions/78.html)
More Info: https://bandit.readthedocs.io/en/1.9.4/blacklists/blacklist\_imports.html#b404-import-subprocess
Location: .\app_fixed.py:94:4
93
94     import subprocess
95     result = subprocess.run(allowed_commands[choice], capture_output=True, text=True)

-----
>> Issue: [B603:subprocess_without_shell_equals_true] subprocess call - check for execution of untrusted input.
Severity: Low Confidence: High
CWE: CWE-78 (https://cwe.mitre.org/data/definitions/78.html)
More Info: https://bandit.readthedocs.io/en/1.9.4/plugins/b603\_subprocess\_without\_shell\_equals\_true.html
Location: .\app_fixed.py:95:13
94     import subprocess
95     result = subprocess.run(allowed_commands[choice], capture_output=True, text=True)
96     return f"<pre>{result.stdout}</pre>"

-----

Code scanned:
    Total lines of code: 78
    Total lines skipped (#nosec): 0

Run metrics:
    Total issues (by severity):
        Undefined: 0
        Low: 2
        Medium: 0
        High: 0
    Total issues (by confidence):
        Undefined: 0
        Low: 0
        Medium: 0
        High: 2
Files skipped (0):
```

Only two Low severity, informational warnings remain, both related to the use of Python's `subprocess` module in the `/run` endpoint. These were manually reviewed and confirmed safe: the endpoint only executes commands from a fixed whitelist dictionary (`allowed_commands = {"date": [...], "whoami": [...]}`), so user-supplied input can never reach the shell directly. Bandit cannot infer this context from the code pattern alone — it flags any `subprocess` usage as a caution regardless of how it's guarded. This is a clear example of why automated tool output should be interpreted with manual judgment, not accepted or dismissed blindly.